M T

True Tech Hack 2026

LOCALSCRIPT: ЛОКАЛЬНЫЕ АГЕНТСКИЕ СИСТЕМЫ КАК КЛЮЧ К БЕЗОПАСНОМУ РАЗВИТИЮ БИЗНЕСА

C

©2025 Cloud.ru Любое копирование и воспроизведение содержания (в том числе частичное) без разрешения правообладателя запрещено.

Внедрение AI-агентов повышает эффективность бизнеса, но зависимость от внешних сервисов создаёт риски утечки данных и приводит к финансовым убыткам

КОНТЕКСТ

Агентские системы позволяют бизнесу:

- Ускорять разработку и вывод продуктов на рынок
- Снижать рутину и освобождать инженеров для сложных задач
- Масштабировать экспертизу без найма десятков разработчиков

ПРОБЛЕМА

Во многих интеграционных и безопасностных контурах нельзя использовать внешние AI-сервисы:

✕ Риски утечки данных и коммерческой тайны

✕ Зависимость от вендоров (изменение цен, политик, доступности)

✕ Нарушение требований к инфраструктуре и комплаенсу

✕ Большие модели требуют дорогих ресурсов и плохо подходят для локального развертывания

ВЫВОД

Необходима агентская система, которая:

✓ работает на собственной инфраструктуре

✓ использует легковесную локальную модель

✓ понимает запрос пользователя на естественном языке

✓ помогает создавать корректный Lua-код без отправки данных наружу.

Мы создали не просто сервис генерации кода, а полноценный управляемый пайплайн

1. Управляемый пайплайн

анализ задачи → выбор шаблонов → генерация → валидация → исправление

Executive Summary

2. Многоуровневая валидация

синтаксис Lua, доменные ограничения и runtime-проверка

3. Автоматическое восстановление

при ошибке система уточняет запрос и генерирует заново

4. Адаптация под LowCode

отдельные правила и ограничения для MWS Octapi

5. Механизм уверенности

числовая оценка качества и причины вывода

6. Диалоговый режим

уточняющие вопросы, память сессии, diff между версиями

7. Полная локальность

Docker-контур: backend + фронт + Ollama, без внешних API

8. Готовность к продакшну

воспроизводимость, предсказуемость, контроль данных

Интерфейс взаимодействия с агентом – не просто «запрос», а настоящий чат-бот

Пользователь формирует задачу на естественном языке

↓

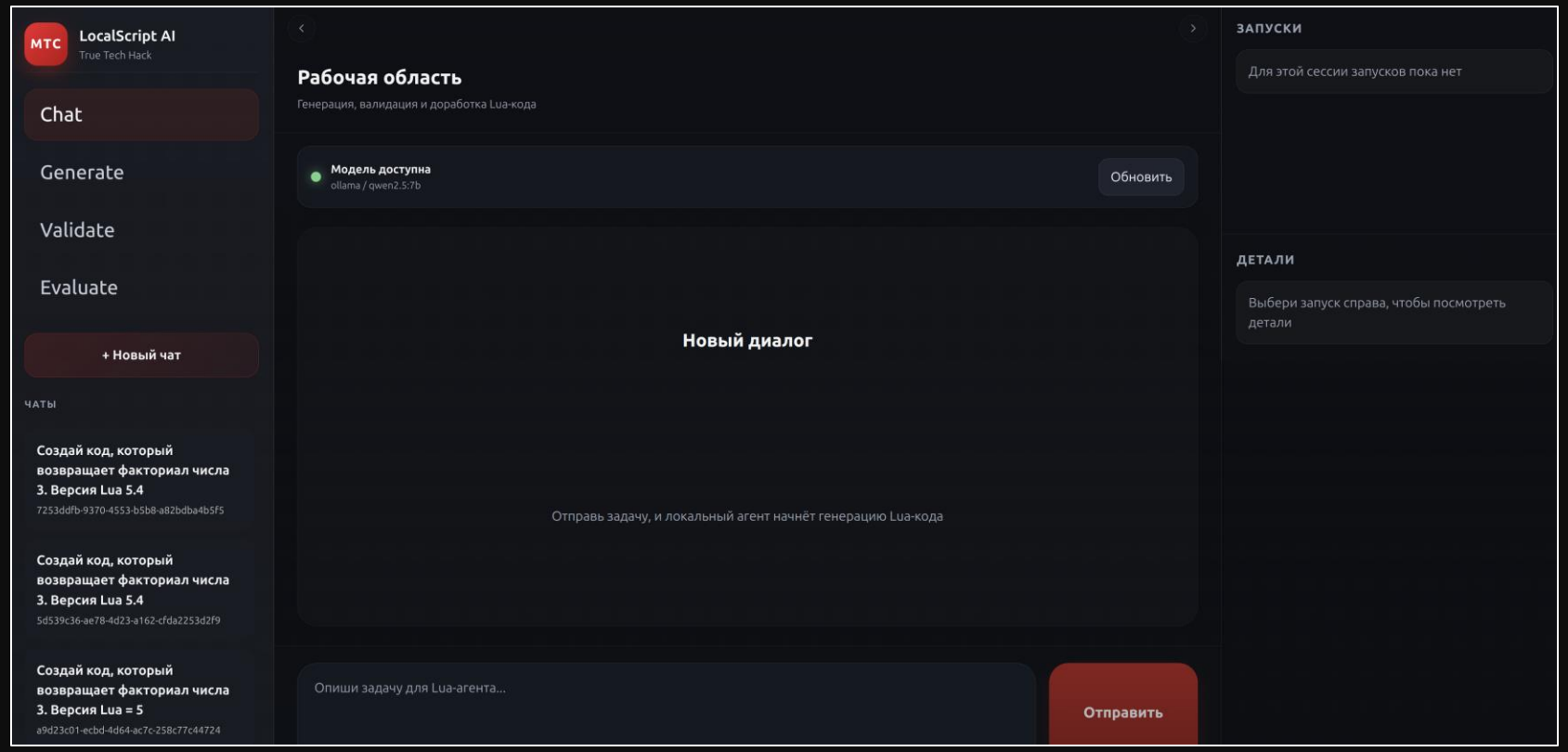
Система:

- Анализирует постановку
- Определяет формат результата
- Выбирает стратегию

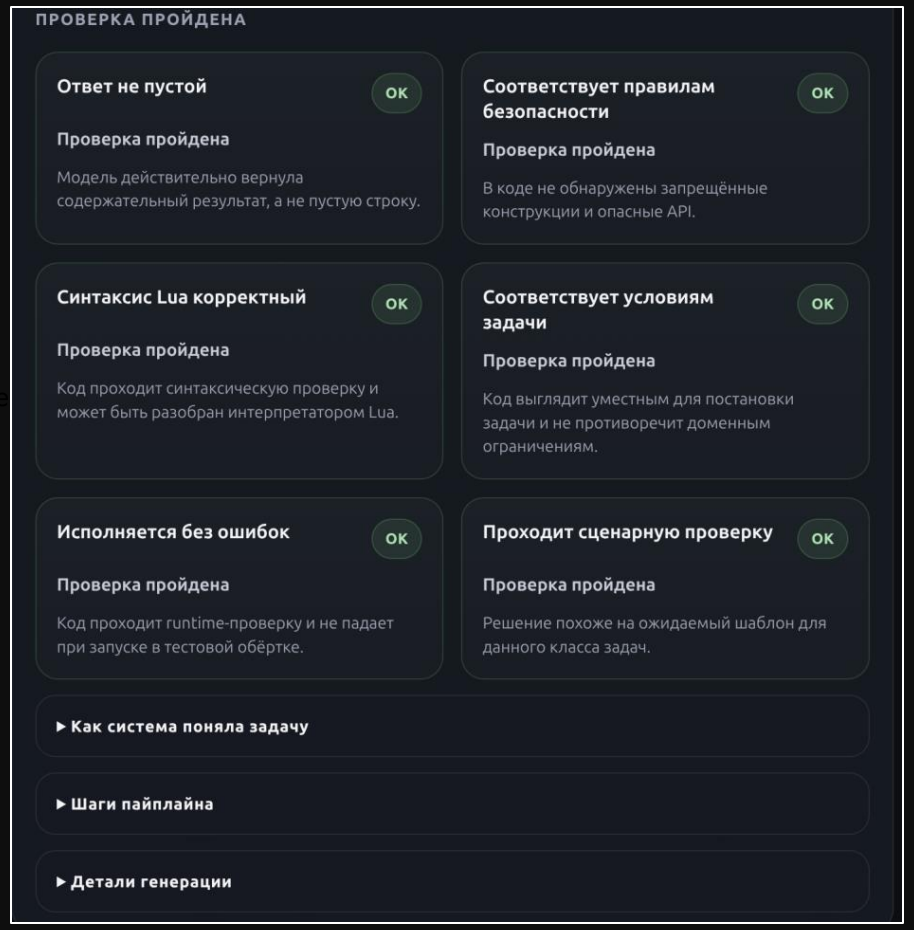
↓

Результат - готовый Lua-код

Ключевая особенность: пользователь не думает о формате — система сама его определяет



Отличительная особенность нашего решения – повышенная внимательность к валидации результата



Каждый сгенерированный код проходит автоматическую проверку:

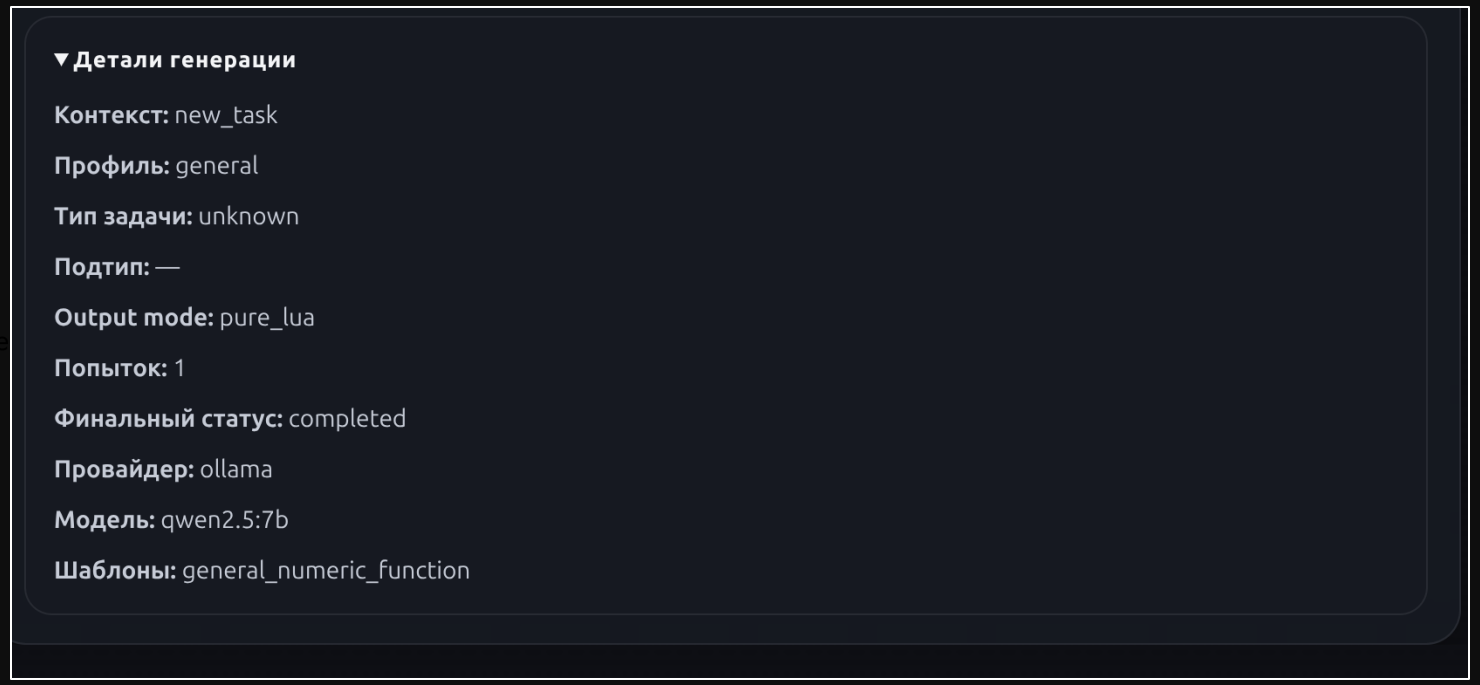
- Проверка на пустоту результата
- Проверка безопасности
- Проверка синтаксиса Lua
- Проверка соответствия задаче

После этого система:

- **Рассчитывает уровень уверенности**
- **Объясняет, почему результат корректен**

Это делает поведение модели контролируемым и проверяемым

Для разработчиков легко изменять агента и делать его более гибким, так как наш проект обеспечивает прозрачность принятия решений



Система сохраняет полный контекст генерации:

- тип задачи
- выбранный профиль
- режим вывода
- использованные шаблоны
- провайдер и модель

Это позволяет:

✓ анализировать ошибки
✓ воспроизводить результат
✓ контролировать поведение системы

Для пользователей модели весь процесс генерации прозрачен – от анализа задачи до времени выполнения

context_resolution 0 ms	OK
task_analysis 0 ms	OK
output_mode_selection 0 ms	OK
template_selection 0 ms	OK
generation 2951 ms	OK
validation_attempt_1 7 ms	OK
scenario_validation_attempt_1 0 ms	OK
output_formatting 0 ms	OK

Каждая генерация разбивается на этапы:

- анализ задачи
- Выбор формата
- генерация
- валидация
- форматирование

Для каждого шага фиксируются:

- статус
- время выполнения

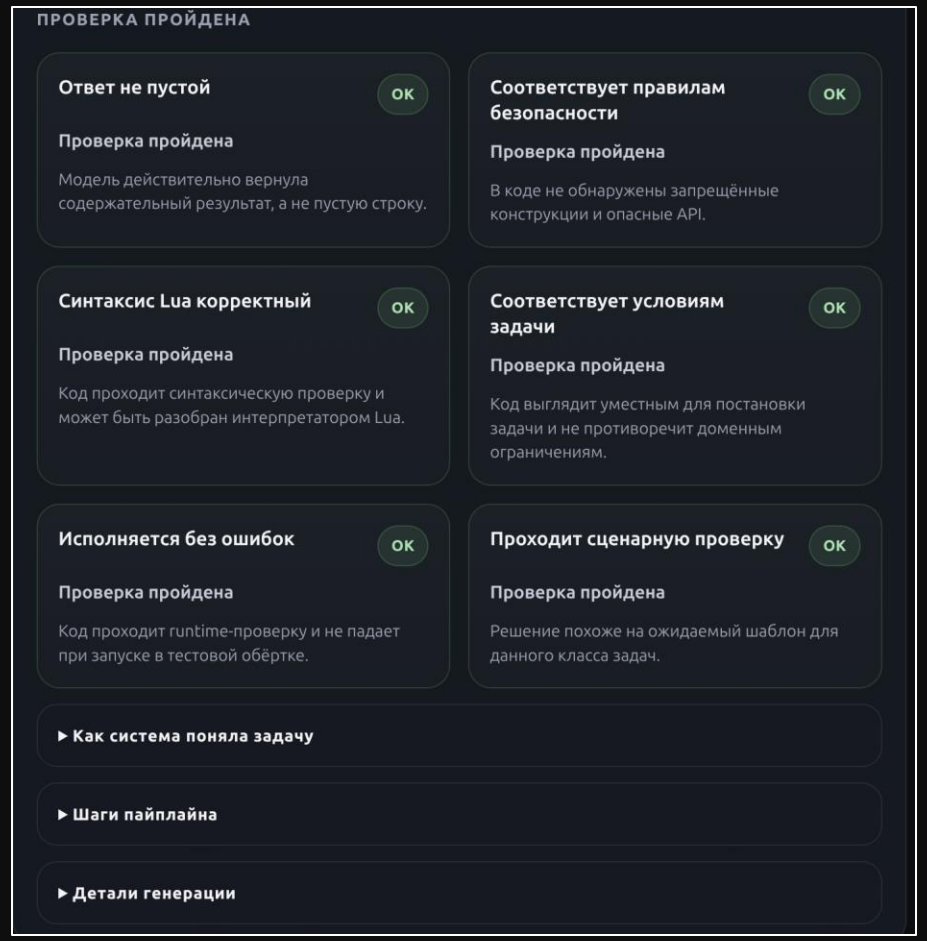
Такой пайплайн даёт возможность пользователю понимать, какой путь прошёл агент при генерации итогового кода.

На выходе пользователь получает не просто исходный код, а визуально понятные блоки и уровень уверенности модели в ответе

Результат проверки генерации представлен в виде понятных блоков:

- ✓ что прошло успешно
- ✓ где потенциальные риски
- ✓ почему код считается корректным

Визуализация в виде блоков важна, так как на выходе мы объясняем полученный результат, а не просто выдаём его.



Итоговый код не требует ручных правок и выполняет все исходные задачи, запрошенные пользователем

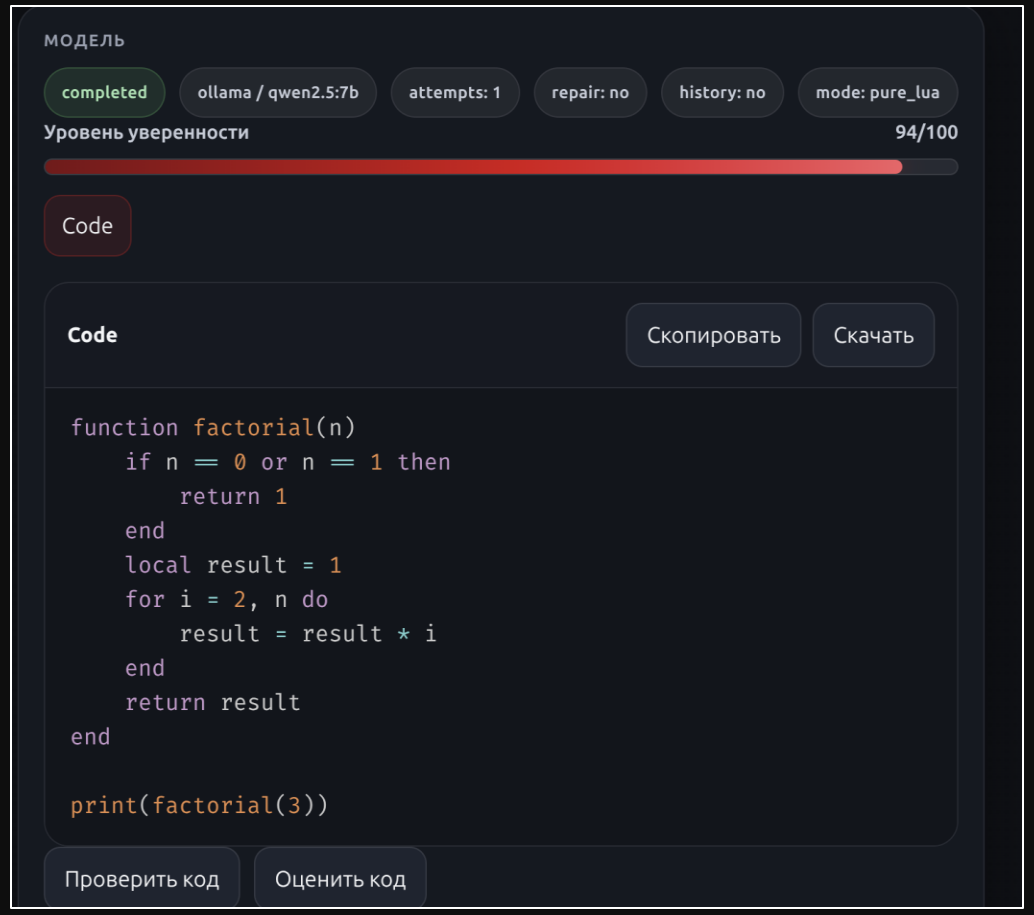
Пользователь получает:

- чистый Lua-код в различных форматах
- решение запроешнной им задачи
- возможность не переписывать код

Executive Summary

Дополнительно:

- возможность повторной валидации
- оценка качества
- сохранение результата



Наше решение ориентировано на локальный запуск, поэтому оно имеет ряд ограничений

Ограничения модели

- локальная модель (qwen2.5:7b) уступает большим облачным моделям
- возможны неточности в сложной логике и редких кейсах
- ограниченный контекст

Ограничения валидации

- проверка выполняется в изолированной среде
- покрывает базовые сценарии, но не все edge-кейсы
- нет полного покрытия пользовательской логики

Инфраструктурные ограничения

- требуется Docker и ресурсы под локальную модель
- inference latency выше, чем у облачных API
- масштабирование требует настройки окружения

Мы сознательно выбрали компромисс в пользу локальности и безопасности

Валидация снижает риск, но не заменяет тестирование в продакшене

Решение ориентировано на локальный запуск